0.1 Heuristics

The heuristics suggested here are based on the idea, that the prior is able to express an inherent certainty that can be leveraged to adjust the amount of simulation needed to generate a successful training sequence while lowering the computational efforts.

0.1.1 Entropy of Likeliest k Actions

The goal for this heuristic is to calculate the entropy of the k highest-rated moves of the predictor's move probability distribution and adjust the amount of simulations accordingly.

Entropy is by itself a measure of uncertainty for a distribution of probabilities. The use of the entropy is therefore an apparent possible solution to the goal of this thesis. Furthermore, using the 2 likeliest options of a probability distribution is a common approach to gather a measure of certainty of that distribution. Increasing this window of consideration should give a more balanced view.

The important formulas are

$$H_{\text{norm}} = \frac{H(\text{Top}k)}{\ln(k)}, \tag{1}$$

$$n_{\text{new}} = n_{\text{base}} \cdot \frac{2}{1 + e^{-s(H_{\text{norm}} - sp)}} \tag{2}$$

where

- $H(\text{Top}k)$ is the entropy of the k highest ranked moves of π that are sorted descendingly in $\text{Top}k$.
- H_{norm} is the result of re-normalising the entropy of these moves to within the range $[0, 1]$,
- n_{new} is the applied number of simulations,
- n_{base} is the baseline number of simulations,
- s is the steepness of the graph, and
- sp is the position of the inflection point on the x-axis.

Equation 2, represented in Figure 1, is a logistic function. The formula here will continue to be used with $s = 8$ and $sp = 0.5$. The graph is maximised at $\frac{2}{1+e^{-8(1-0.5)}} \approx 1.964$ and, because H is normalised, it is minimised at $\frac{2}{1+e^{-8(0-0.5)}} \approx 0.036$. These values should allow the entropy of the k highest-rated moves to transition fluidly around the inflection point at 0.5.

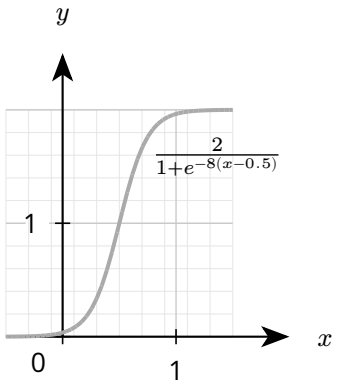

Figure 1: Logistical uncertainty function

Effort for this heuristic should be negligible. The algorithmic complexity of finding the highest-sorted $\mathrm{O}(|A| \cdot \log k)$, with $|A|$ as the space of all actions. There also is the additional cost of calculating the entropy of $\mathrm{Top}k$.

Making use of more values of that distribution should also provide a clearer estimation of certainty, differences are discussed in a later section.

0.1.2 Comparison of High And Low Ranking Moves

This heuristic measures the disparity between the probability densities of the k highest-rated moves and the $A - k$ lowest-rated moves. Using this distance, the number of simulations is adjusted.

The above heuristic has a strong focus on relatively few high-rated moves compared to the overall space of possible moves. This increases vulnerability to overconfidence where it is not warranted. That is especially the case when a single move is disproportionately well-rated. Therefore, one solution to this problem can be a more global consideration of the move probability space.

$$d = \sum_{a \in \mathrm{Top}k} \pi(a) - \sum_{a \in A \setminus \mathrm{Top}k} \pi(a), \tag{3}$$

$$n_{\mathrm{new}} = n_{\mathrm{base}} \cdot (\frac{2}{)}, \tag{4}$$

where

- $\pi(a)$ is the probability of a in π,
- $\mathrm{Top}k$ is the array of the highest-rated k actions,
- A is the space of all possible actions.

Equation 4 is the same as Equation 2 with the entropy exchanged for the difference of the probability densities.

This solution should be able to help mitigate the inequal consideration of moves from the first heuristic. Computational effort consists of the sorting in $O(n \log k)$ and the addition. Therefore the effort should be about as much as for the first heuristic.
Differences in k here will also be discussed in a later section.

0.1.3 Auxiliary Head Estimating Uncertainty

Lan et al. introduced supplementary networks to estimate certainty and stop searching when reaching a certain threshold. However, as merging networks for predicting value and policy has proved successful previously, enabling the network to predict its own prediction-search disagreement should also turn out beneficial. The predictor network f on board position s will return

$$f(s) = (\mathbf{p}, v, u), \tag{5}$$

where $\mathbf{p}$ is the move probability distribution, v the evaluation, and u the uncertainty estimation. The number of simulations is adjusted according to the formula

$$n_{\text{new}} = n_{\text{baseline}} \cdot \exp(u). \tag{6}$$

This means, that searching more thoroughly correlates with higher uncertainty while a lower number of simulations correlates with less uncertainty. The exponential function allows both to increase and decrease the number of simulations. n_{new} also is limited to a maximal number to stop the search from going unendingly long.
The loss will be calculated using the expression

$$l_{\text{all}} = l_{\text{az}} + c \cdot (u - \lambda \cdot \tau(\mathbf{p}, \pi))^2, \tag{7}$$

where

- l_{az} is the error signal used by AlphaZero considering $(\mathbf{p}, \sigma, v, \theta))$,
- c is a weighting constant for MSE,
- $\tau(\mathbf{p}, \pi)$ is the KRCC of board state s,
- λ is a scaling constant for the KRCC, and
- π is the distribution found by MCTS.

In this context $\tau(\mathbf{p}, \pi)$ measures the similarity between the ranked move distribution estimated by the prior and the ranked move distribution determined by MCTS.
This heuristic should be capable of reducing the average simulation numbers by predicting how much move estimations change during MCTS in different positions. However,

especially the process can be very time-intensive in the beginning of a single game when more moves are available and the calculation of KRCC requires more effort.

While the idea is promising, a glaring problem with this approach is the computational cost of calculating the KRCC for the two distributions. One solution is to similarly to the above heuristics consider only the highest k actions. This is also looked at in the following sections.

0.1.4 Random

Following the example of Wu et al. [1], a uniformly chosen number from $n \in [1, 300]$ will be used as the number of playouts for MCTS in order to investigate in comparison whether these strategies actually fulfill their purpose and have an active part in lowering computational effort. If randomly adjusting the amount of simulations for MCTS performed equally well as the strategies suggested above and assuming that they all perform similarly well to the base line model, this would suggest that the strategies constitute unnecessary computational effort.

Bibliography

[1] D. J. Wu, “Accelerating Self-Play Learning in Go,” Nov. 10, 2020. doi: 10.48550/arXiv.1902.10565.